



A geocoder that lives inside PostgreSQL

Search, autocomplete and reverse geocoding for the State of Maine
PostgreSQL 18 · PostGIS · core and contrib extensions only

`pg` + *geo* — said "pee-geo"

Measured against Pelias on the same data · full study in `docs/REPORT.md`

Why build it

A self-hosted geocoder was needed for one state, a handful of users, and a dispatch application whose client addresses are sensitive.

- Commercial APIs charge per request and see every address you look up
- Pelias is the established open-source answer: accurate, fast, and six services plus Elasticsearch
- The guiding principle was G6: the fewest components, data staying in PostgreSQL — unless that produces an inferior product

So the question was not "can it be done in SQL?" but "is the result worse?" pgeo was built to answer that honestly, with Pelias as the reference and the test oracle.

Architecture

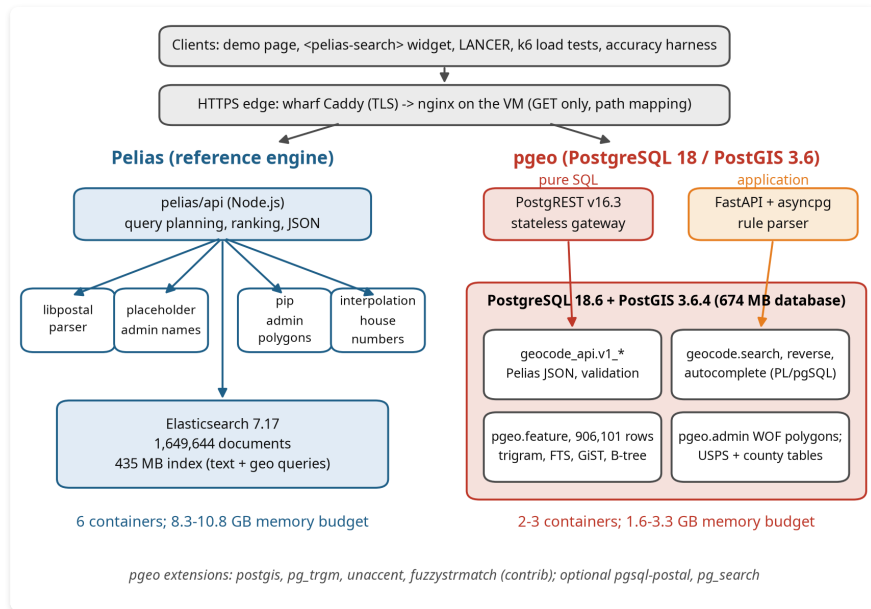
Everything that ranks, filters and formats is SQL inside one database.

- Data: one feature table, 906,101 rows, 654 MB with indexes
- Query pipeline: PL/pgSQL — normalize, parse, candidates, filter, score, dedupe, JSON
- HTTP: a stateless gateway, no application code

Two front ends, same SQL:

PostgREST "pure SQL" — maps `/v1/*` to functions

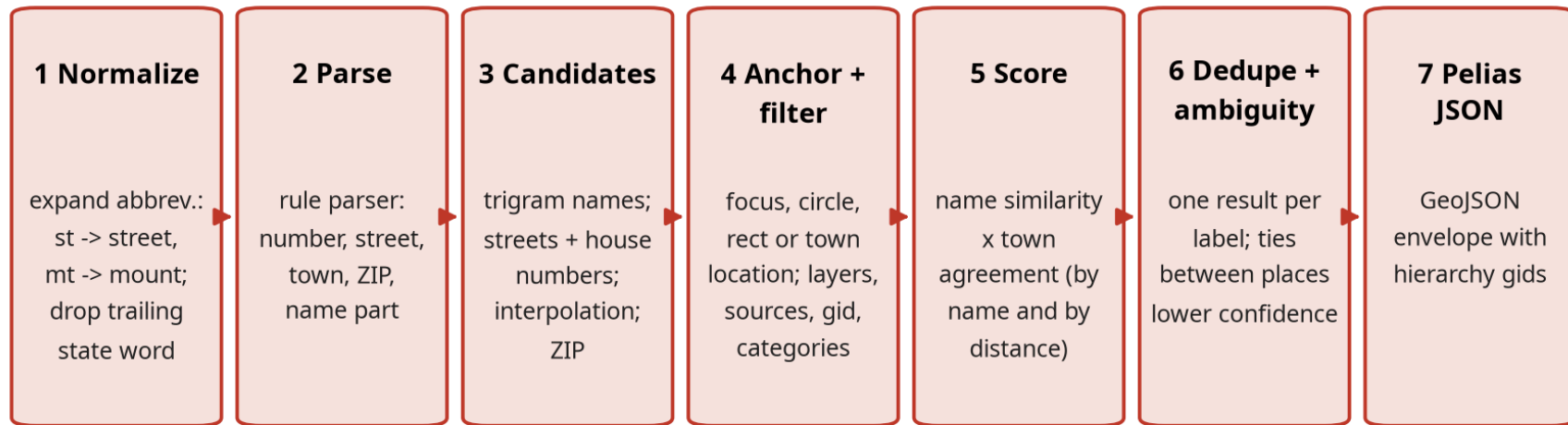
FastAPI thin Python service, same functions



Pelias (left) coordinates six processes; pgeo (right) is two containers.

The query pipeline

Every step runs inside PostgreSQL; the FastAPI and PostgREST front ends pass parameters in and JSON out



Each step is plain SQL in a file under version control, so a ranking change is a pull request and the accuracy gate measures it. Tuning lives in steps 1, 3, 4, 5 and 6.

What it needs

To run

Containers	2 (PostgreSQL, PostgREST)
Floor, 3 users	0.25 vCPU, 1.40 GB
Confirmed on a real VM	1 vCPU / 1 GB, 32 users
Disk	654 MB of tables; size for 3-4x
Extensions	<code>postgis</code> , <code>pg_trgm</code> , <code>unaccent</code> , <code>fuzzystrmatch</code> — all core or contrib

To build

Wall time	13 min
Peak memory	5.5 GB
CPU	1.2 cores average
Result to ship	a 117 MB dump

The build needs four times the runtime floor. The cheap VM that runs pgeo cannot build it — build on a workstation, ship the dump.

Where it flexes

- Two HTTP front ends over identical SQL: PostgREST for no application code, FastAPI when you want Python in the path
- Five tuning profiles (`tiny` → `workstation`), applied and verified against the running server; `pgeo-tune auto --cpus N --mem-gb M` sizes anything between
- Three parsing modes: a rule parser in SQL (most accurate here), libpostal as a service, or as an extension
- Subset builds — load only the sources you want
- It is just a database: replicas, `pg_dump` , PITR, your existing backup and monitoring all apply
- Deployable from a 1 GB VPS to a multi-core VM without changing anything but the profile
- Deployable without Pelias at all: `pelias_enabled=false` and pgeo answers on the canonical `/v1/*` paths, so an existing Pelias client needs no change and the demo page drops its engine switch and Compare tab by itself

The flexibility that matters most: no new operational vocabulary. If you run PostgreSQL, you already run this.

The alternatives inside PostgreSQL

pgeo is not the first geocoder in a database. Surveyed September 2026:

Project	Logic	Data	Interface
PostGIS TIGER Geocoder	all PL/pgSQL	US Census TIGER only	SQL only, no HTTP
osmgeocoder	mostly SQL, Python orchestration	OSM, optionally OpenAddresses	Python library, optional Flask
Nominatim	indexing in PL/pgSQL, search in Python	OSM	HTTP
pgeo	all SQL	OA, OSM, WOF, GNIS, ZCTA, Overture	HTTP, no application code

Measured against three other engines

Same machine, same two cores, same corpus, ramp, targets and 1,560 accuracy cases.

Two cores	Pelias	Photon	Nominatim	pgeo	pgeo, optimized
Concurrent users	192	128	64	48	64
Autocomplete p95 at 48 users	21 ms	53 ms	85 ms	103-189 ms	82 ms
Memory under load	8.4 GB	1.84 GB	2.19 GB	2.5 GB	2.5 GB
Accuracy, 1,560 cases	75.5%	73.7%	61.7%	95.8%	95.8%
Autocomplete accuracy	86.7%	58.0%	8.0%	95.3%	95.3%
Data sources	6	OSM only	OSM only	6	6

Why the accuracy column is not the whole story

- Photon and Nominatim index OpenStreetMap only. They structurally cannot hold the 780,260 OpenAddresses E911 points or the 76,582 Overture places the test set is built from — Photon scores 19% on venues, Nominatim 9%, because those places are not there to find
- Nominatim scores 8% at autocomplete with 75% of prefixes returning nothing. It has no autocomplete endpoint — which is precisely why Photon exists
- Nominatim is exact when it answers: median error 0 m on addresses. Its failure mode is recall, not precision — it returns nothing for 39% of all queries
- Neither expresses uncertainty. No confidence score, so neither can say "here is a result, but I do not believe it" — disqualifying for dispatch, independent of speed

What pgeo can honestly claim against the OpenStreetMap engines is not that it searches better, but that it can hold data they cannot. What survives regardless of data is the capacity result.

A third more capacity, no answer changed

The query path had never been profiled. Four fixes, none altering what a query means:

			Was	Now
A filter function that filtered nothing, called per row			127 ms	48 ms
Result records built for 376 rows, 10 kept			42 ms wasted	built for 10
Hot columns behind 20 cold ones in a 780-byte row			7,809 pages	2,912
s:* expanded to every token starting with "s" — including <i>street</i>			44.8 ms	2.9 ms
Users within targets	1 vCPU	2 vCPU	4 vCPU	all cores
Before	24	48	96	128
After	32	64	128	192

Features

Pelias-compatible API — 28 of 28 contract cases

- `/v1/search` , `/v1/search/structured`
- `/v1/autocomplete` , `/v1/reverse`
- `/v1/place` , `/v1/attribution`

Filters: layers, sources, focus point, rectangle, circle, country, `boundary.gid` , categories

Beyond Pelias

- `/v1/address` — USPS Publication 28 structured addresses
- nearest street address for a venue that has none

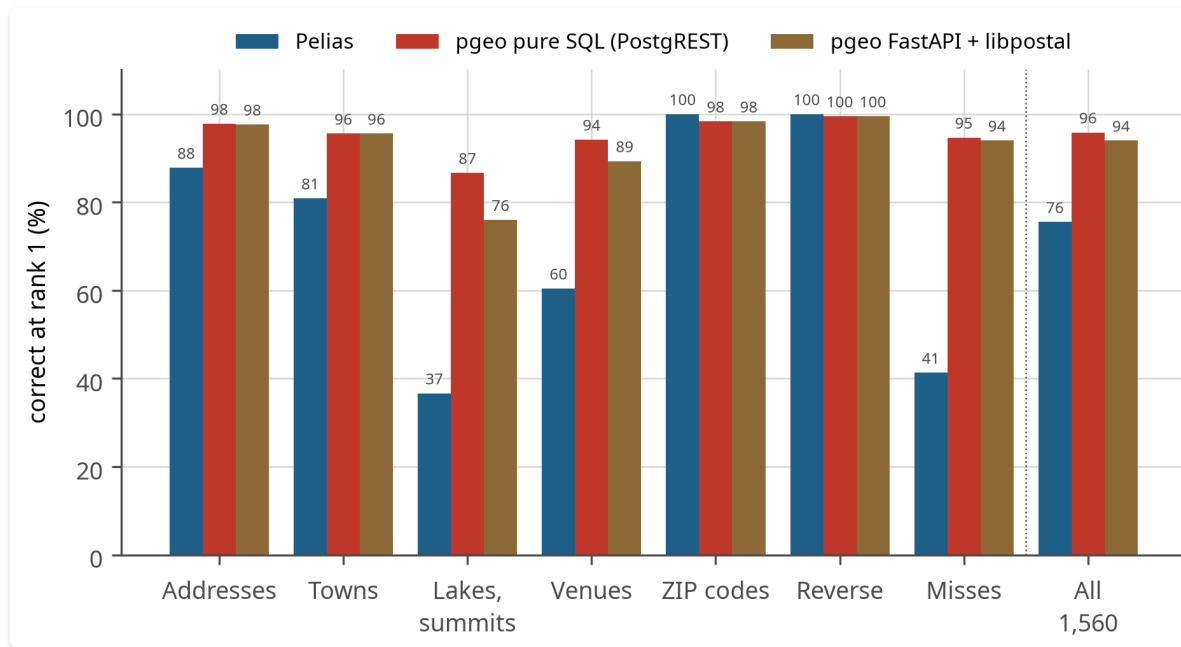
Built for messy input

- trigram similarity on whole names, not tokens
- abbreviations expanded before comparing
- the queried town treated as a *location*, not just a name
- confidence lowered for ties between distinct places

Operations

- accuracy gate on every build
- tuning profiles verified against the server
- one-command rebuild

How accurate



95.8%

overall (Pelias 75.5%)

87%

typos (Pelias 30%)

95%

misses handled (41%)

3-4×

Pelias throughput per CPU

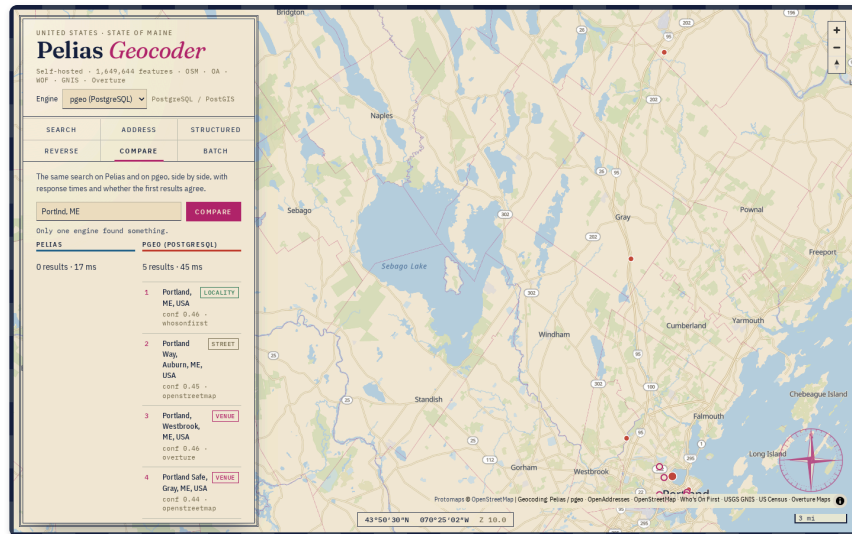
1,560 engine-independent cases whose answers come from the source data.

The trade, in one screenshot

Searching the typo "Portlnd, ME" on both engines, side by side in the demo app:

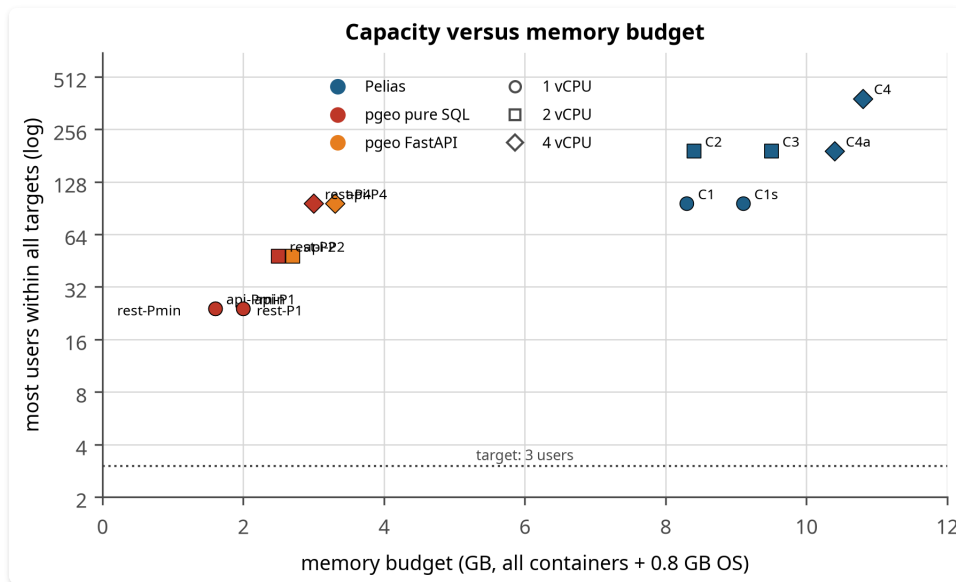
	Results	Time
Pelias	0	17 ms
pgeo	5, Portland at rank 1	45 ms

pgeo answers what people actually type. Pelias is faster at answering nothing.



That is the whole study: pgeo trades throughput per CPU for accuracy on real input, and for a quarter of the memory.

How small a server



Both engines serve three concurrent users on a **quarter of a vCPU**.

Memory decides:

Confirmed on a real 1 vCPU / 1 GB VM: the three-user load ran at **21% of its latency budget**, using 307 MB, and held **32 concurrent users**.

Floor

Plan

Pelias's floor is set by services that will not start below a threshold — libpostal needs 2 GB before it serves anything.

Using it

```
# Search
curl 'http://host/v1/search?text=389+Congress+St,+Portland,+ME&size=5'

# Autocomplete, biased to the map centre
curl 'http://host/v1/autocomplete?text=389+congres&focus.point.lat=43.656&focus.point.lon=-70.26'

# Reverse
curl 'http://host/v1/reverse?point.lat=43.6568&point.lon=-70.2626'

# A structured USPS address for whatever the user picked (pgeo only)
curl 'http://host/v1/address?ids=openstreetmap:venue:node/123456'
```

Responses are GeoJSON with the Pelias `geocoding` block, so an existing Pelias client needs no changes. Point it at the new base URL.

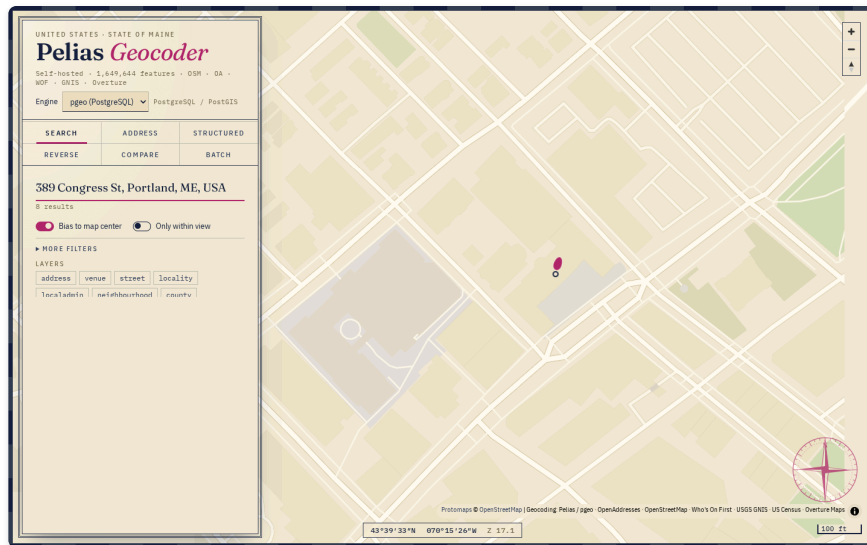
```
scripts/pgeo_setup.sh --profile workstation # secrets, images, containers, edge
scripts/pgeo_rebuild.sh --profile workstation # load, tune, verify, accuracy gate
```

The demo app

One page, both engines, switchable at the top.

- Search with map-centre bias and filters
- Address — type-ahead, then a formatted USPS address
- Structured, Reverse, Batch (CSV)
- Compare — the same query on both engines

Served same-origin with a strict CSP: no CDN, no remote script, every value written with `textContent`.



Limitations

- **Throughput:** 3-4× fewer concurrent users per CPU than Pelias. Plan ~24 users per vCPU, then halve it for headroom
- **One state, tested.** Maine only – and measured: across Maine's own data range pgeo loses four times its capacity where Pelias loses two, so coverage growth costs pgeo roughly twice as much. Multi-state is still untested
- **English only** — `lang` is accepted and ignored
- **US addresses only** for `/v1/address` (USPS Publication 28)
- **Our own code:** the ranking SQL has one project's eyes on it, where Pelias has many
- **Tuned against the set that scores it** — treat the 20-point accuracy gap as an upper bound until both engines meet queries neither has seen
- **No authorization yet:** anything that reaches the edge can query

Autocomplete is the bottleneck, not reverse geocoding — it has the tightest target (250 ms) and fires several times per typed word. That is where the next factor of two lives. Measured against Photon, the type-ahead specialist: two to three times slower per

Security, briefly

In pgeo's favour

- the data store authenticates; the API role holds `SELECT` only
- two components to patch, not six
- no query parameters in the PostgreSQL log

Against

- the ranking SQL and edge rules are this project's own code

Shared posture

- every engine port on loopback; only the edge listens
- `GET` only, per-client rate limits, strict CSP
- 30 posture tests run on every build and deploy

Neither engine authorizes callers yet. On a LAN that is a decision; on the internet it would be a defect.

A finding worth repeating: PostgreSQL was configured not to log query parameters, but the nginx access log in front of it recorded the full query string — every address searched, in plaintext. A control at one layer, undone by a default at the next. Fixed, and now tested.

When to use which

Scenario	Use	Why
A few users, small VM or VPS	pgeo	meets every target from 1.4 GB and a quarter core
Messy input: typos, venues, half-remembered names	pgeo	87% vs 30% on typos
Structured US addresses for another system	pgeo	/v1/address
Hundreds of concurrent users per vCPU	Pelias	3-4× the throughput
Multi-state, national, international	Pelias	built for planet scale
Fewest components, data in PostgreSQL	pgeo	a database and a stateless gateway

Read the study

The full comparison — methods, 20 figures, 37 tables, and every number computed from saved results — is in the repository.

<code>docs/REPORT.md</code> / <code>.pdf</code>	the study
<code>docs/REBUILD.md</code>	rebuild everything, one command per stage
<code>docs/SECURITY_DEPLOYMENT.md</code>	deployment tiers and their trade-offs
<code>docs/PELIAS_COMPATIBILITY.md</code>	what matches Pelias, and what does not
<code>docs/ADDRESS_API.md</code>	the USPS address endpoint

Measured against Pelias, built at Mapzen from 2013 and maintained since 2018 by Geocode Earth under the MIT licence. It supplied the API pgeo implements and the reference answers that defined "correct". Where pgeo comes out ahead, it is ahead on a problem Pelias defined.